

Giáo Án

Ghi chú cho giảng viên:

- Ở giai đoạn này, học viên chỉ cần hiểu hiểu và vận dụng được lý thuyết, chưa cần tối ưu code.
- Mục tiêu là giúp học viên **viết được code chạy được trước**, chưa cần viết code ngắn gọn hay tối ưu ngay từ đầu (clean code).
- Giảng viên có thể sử dụng nhiều ví dụ thực tế để minh họa, tránh giải thích quá phức tạp gây khó hiểu.
- Nếu cảm thấy học viên chưa rõ lý thuyết, vui lòng **tách nhỏ nội dung** thành các kiến thức dễ tiêu thụ hơn.
- Nếu học viên gặp khó khăn khi làm bài tập, vui lòng **chia bài tập lớn thành các bài tập bổ trợ nhỏ** hoặc **tự bổ sung thêm bài tập tương tự** để giúp họ từng bước hoàn thành.

JSX và Component

1. Mục tiêu

- Hiểu JSX là gì và vai trò của nó trong React.
- Biết cách sử dụng JSX để viết giao diện trong React.
- Hiểu khái niệm Component trong React.
- Biết cách tạo một Function Component.

2. Nội dung bài giảng

2.1 JSX là gì?

- Phần mở rộng cú pháp JavaScript cho phép viết HTML trong JavaScript.
- Không phải HTML thật, mà là cú pháp để React hiểu và tạo UI.
- JSX giúp việc code giao diện (UI) trở nên dễ dàng hơn rất nhiều so với việc viết code JavaScript thuần để tạo giao diện."

2.2 Cú pháp JSX

Giới thiệu cú pháp JSX sơ qua dựa trên file `App.jsx` :

- Cấu trúc Component:** `function App() { ... }`.
- Thẻ JSX:** Mô phỏng giống thẻ HTML: `<div>`, `<a>`, `<img>`, `<h1>`, `<p>`, `<button>`.
- Thuộc tính JSX:** Mô phỏng giống thuộc tính các thẻ HTML. Tuy nhiên sẽ có 1 vài chỗ khác: `className` (thay `class`), `htmlFor` (thay `for`), camelCase tên các sự kiện (`onClick`).
- Biểu thức JavaScript:** `{ }` để nhúng code JS (giá trị, biến, biểu thức) vào JSX.
- JSX chỉ trả về một phần tử gốc:** Một Component JSX chỉ được trả về một phần tử gốc duy nhất. Nếu muốn trả về nhiều phần tử, cần bọc chúng trong một thẻ `<React.Fragment>` hoặc thẻ `<>` rỗng.

2.3 Component là gì?

- Component là các thành phần trong giao diện React.
- Component giúp tái sử dụng, dễ quản lý code.
- Có hai loại Component chính:
 - Function Component:** Sử dụng hàm, đơn giản và dễ hiểu (Được sử dụng trong khoá học)
 - Class Component:** Sử dụng class (ít được dùng trong React hiện đại).

2.4 Khởi tạo Function Component

- Tạo file `Counter.jsx` bên trong `src/` với nội dung như sau:

```
function Counter() {
  return (
    <div>
      <p>Đếm: 1</p>
      <button>Tăng</button>
    </div>
  );
}

export default Counter;
```

- **Sử dụng Component:**

- Xoá toàn bộ nội dung JSX trong `App.jsx` sau đó import `Counter` vào:

```
import './App.css';
import Counter from './Counter';

function App() {
  return (
    <Counter />
  )
}

export default App;
```

Style, Biến, State và Sự Kiện

1. Mục tiêu

- Biết cách áp dụng CSS vào Component trong React.
- Hiểu state là gì và vai trò của nó trong React.
- Biết cách sử dụng `useState` để quản lý trạng thái.
- Hiểu cách bắt sự kiện trong React.

2. Nội dung bài giảng

2.1 Áp dụng CSS vào Component

- **Inline Style:**

- Sử dụng thuộc tính `style` để thêm style trực tiếp vào element.
- Giá trị của `style` là một object JavaScript, với các thuộc tính CSS viết theo camelCase.

Ví dụ trong component `Counter` :

```
function Counter() {
  return (
    <div>
      <p style={{ color: 'red', fontSize: '20px' }}>Đếm: 1</p>
      <button>Tăng</button>
    </div>
  );
}

export default Counter;
```

- Không khuyến khích sử dụng inline style. Tại vì khó quản lý style phức tạp, không tái sử dụng được style.

- **File CSS truyền thống**

- CSS Files là các tệp tin riêng biệt, chứa mã CSS để định nghĩa style cho các phần tử HTML. Trong React, chúng ta có thể tạo các CSS Files này và liên kết chúng với các component để áp dụng style.
- **Tái sử dụng Style:** Các style được định nghĩa trong CSS Files có thể được tái sử dụng cho nhiều component khác nhau trong ứng dụng.
- **Dễ quản lý Style:** Style được tách biệt khỏi code JavaScript, giúp code dễ đọc và dễ bảo trì hơn. Khi dự án lớn, việc quản lý style trong các file riêng biệt sẽ trở nên cần thiết.
- **Vấn đề:** Trong CSS truyền thống, **tên class có phạm vi toàn cục**. Nghĩa là, nếu bạn dùng tên class `.button` ở file CSS nào đó, thì bất kỳ thẻ HTML nào trong toàn bộ ứng dụng của bạn mà bạn gán `className="button"` vào, đều sẽ bị ảnh hưởng bởi style trong CSS đó.

Ví dụ: Chuyển `Counter.jsx` sang file CSS:

- Tạo file `Counter.css` bên trong `src/` :

```
.counter {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 10px;
}

.count {
  color: red;
  font-size: 20px;
}

.button {
  padding: 8px 16px;
  font-size: 16px;
  background-color: blue;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.button:hover {
  background-color: darkblue;
}
```

- Cập nhật `Counter.jsx` để sử dụng File CSS:

```
import './Counter.css';

function Counter() {
  return (
    <div className="counter">
      <p className="count">Đếm: 1</p>
      <button className="button">Tăng</button>
    </div>
  );
}

export default Counter;
```

- **CSS Modules (Nếu học viên đã học React trước đó mới nói thêm phần này):**

- Sử dụng `className` để gán class CSS cho element, tương tự như HTML.

- Có thể sử dụng CSS Modules để quản lý style cục bộ và tránh xung đột tên class.
- Cú pháp: Định nghĩa style trong file `.module.css`, sau đó **import vào component**.

Ví dụ: Chuyển `Counter.jsx` sang CSS Modules:

- Đổi tên file `Counter.css` sang `Counter.module.css`.
- Cập nhật `Counter.jsx` để sử dụng CSS Modules:

```
import styles from './Counter.module.css';

function Counter() {
  return (
    <div className={styles.counter}>
      <p className={styles.count}>Đếm: 1</p>
      <button className={styles.button}>Tăng</button>
    </div>
  );
}

export default Counter;
```

2.2 State trong React

- **State là gì?**

- State (trạng thái) là một khái niệm quan trọng trong React, dùng để quản lý dữ liệu **thay đổi** của một component theo thời gian.
- Khi state của một component thay đổi, React sẽ tự động **render lại** component đó để giao diện người dùng được cập nhật theo dữ liệu mới.
- State giúp component trở nên **động** và **tương tác** hơn.

- **Vai trò của State**

- **Hiển thị dữ liệu động:** State cho phép component hiển thị dữ liệu có thể thay đổi, ví dụ:
 - Giá trị của một bộ đếm (counter).
 - Danh sách sản phẩm được lọc.
 - Trạng thái đóng/mở của một menu.
- **Tương tác người dùng:** State được cập nhật khi người dùng tương tác với component, ví dụ:
 - Nhập liệu vào ô input.
 - Click vào button.
 - Chọn một mục trong dropdown.
- **Quản lý trạng thái giao diện:** State có thể dùng để quản lý các trạng thái hiển thị khác nhau của giao diện, ví dụ:
 - Ẩn/hiện một phần tử.
 - Thay đổi style của một phần tử.
 - Chuyển đổi giữa các tab.

- **Hook `useState`**

- `useState` là một **Hook** trong React, cho phép functional component có thể "nhớ" các giá trị state.
- Để sử dụng `useState`, bạn cần import nó từ React: `import { useState } from 'react';`
- **Cú pháp:**

```
const [state, setState] = useState(initialValue);
```

- `useState(initialValue)`: Khởi tạo state với giá trị ban đầu `initialValue`. Giá trị này có thể là bất kỳ kiểu dữ liệu nào (số, chuỗi, object, array...).

- `state` : **Biến state**, chứa giá trị state hiện tại. Bạn có thể truy cập và hiển thị giá trị này trong JSX.
- `setState()`: **Hàm cập nhật state**, dùng để thay đổi giá trị state. Khi bạn gọi `setState` với một giá trị mới, React sẽ:
 - 1. Cập nhật giá trị của `state`.
 - 2. Render lại component.
- **Ví dụ: Bộ đếm (Counter) sử dụng State**

```
import React, { useState } from 'react'; // Import useState

function Counter() {
  // Khởi tạo state 'count' với giá trị ban đầu là 0
  const [count, setCount] = useState(0);

  return (
    <div className="counter">
      <p className="count">Đếm: {count}</p> {/* Hiển thị giá trị state 'count' */}
      <button className="button">Tăng</button>
    </div>
  );
}

export default Counter;
```

- Trong ví dụ trên, `useState(0)` khởi tạo state `count` với giá trị ban đầu là 0. Biến `count` giữ giá trị hiện tại, và `setCount` là hàm để cập nhật `count`. Hiện tại button chưa có chức năng, chúng ta sẽ thêm sự kiện để button có thể tăng giá trị `count`.

2.3 Sự Kiện trong React

- **Sự kiện là gì?**
 - Sự kiện (event) là các hành động xảy ra trong trình duyệt, thường là do người dùng tương tác với trang web (ví dụ: click chuột, nhấn phím, di chuyển chuột...).
 - Trong React, chúng ta có thể "bắt" các sự kiện này và thực hiện một hành động nào đó khi sự kiện xảy ra.
- **Xử lý sự kiện trong React**
 - Trong React, chúng ta xử lý sự kiện bằng cách sử dụng **thuộc tính sự kiện** (event attributes) trực tiếp trong các phần tử JSX.
 - Các thuộc tính sự kiện có tên theo cú pháp `on[TênSựKiện]`, ví dụ: `onClick`, `onChange`, `onMouseOver`, `onSubmit` ... (chú ý chữ cái đầu của tên sự kiện viết hoa).
 - Giá trị của thuộc tính sự kiện là một **hàm**, hàm này sẽ được gọi khi sự kiện xảy ra. Hàm này thường được gọi là **event handler** (hàm xử lý sự kiện).
- **Ví dụ: Xử lý sự kiện `onClick` để tăng bộ đếm**

```
import React, { useState } from 'react';
import styles from './Counter.module.css';

function Counter() {
  const [count, setCount] = useState(0);

  // Hàm xử lý sự kiện onClick cho button "Tăng"
  const handleIncrement = () => {
    setCount(count + 1); // Cập nhật state 'count' khi button được click
  };

  return (
    <div className={styles.counter}>
      <p className={styles.count}>Đếm: {count}</p>
    </div>
  );
}
```

```
    <button className={styles.button} onClick={handleIncrement}> {/* Gán hàm handleIncrement cho sự kiện
      Tăng
    </button>
  </div>
);
}

export default Counter;
```

◦ **Giải thích:**

- Chúng ta định nghĩa một hàm `handleIncrement` để xử lý sự kiện click của button.
- Trong hàm `handleIncrement`, chúng ta gọi `setCount(count + 1)` để cập nhật state `count`, làm cho giá trị đếm tăng lên 1 mỗi khi button được click.
- Chúng ta gán hàm `handleIncrement` cho thuộc tính `onClick` của button: `<button onClick={handleIncrement}>Tăng</button>`. Lưu ý chúng ta truyền **tham chiếu hàm** `handleIncrement` chứ không **gọi hàm** `handleIncrement()`.

Cập nhật State với Callback

1. Mục tiêu

- Hiểu lý do tại sao cần cập nhật state bằng callback trong React.
- Nắm vững cú pháp và cách sử dụng callback function để cập nhật state.
- Nhận biết được các trường hợp nên sử dụng callback khi cập nhật state.
- Biết cách viết code React an toàn và tránh các lỗi tiềm ẩn liên quan đến cập nhật state bất đồng bộ.

2. Nội dung bài giảng

2.1 Tại sao cần Callback khi cập nhật State?

- **Tính bất đồng bộ của cập nhật State:**
 - Trong React, việc cập nhật state (khi gọi `setState` hoặc hàm `setCount` từ `useState`) có thể là một quá trình **bất đồng bộ**.
 - Điều này có nghĩa là khi bạn gọi hàm cập nhật state, React có thể không thực hiện cập nhật state ngay lập tức, mà có thể đợi đến khi hoàn thành các công việc khác, hoặc gom nhóm nhiều cập nhật state để tối ưu hiệu năng.
 - Vì tính bất đồng bộ này, giá trị của state **ngay sau** khi gọi hàm cập nhật có thể **chưa được cập nhật ngay lập tức**.
- **Ví dụ minh họa vấn đề:**

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  const handleIncrement = () => {
    setCount(count + 1); // Gọi cập nhật state lần 1
    setCount(count + 1); // Gọi cập nhật state lần 2
    setCount(count + 1); // Gọi cập nhật state lần 3
  };

  return (
    <div>
      <p>Đếm: {count}</p>
      <button onClick={handleIncrement}>Tăng 3 lần</button>
    </div>
  );
}
```



```
    </div>
  );
}

export default Counter;
```

- **Mong đợi điều gì khi click button "Tăng 3 lần"?** Có thể bạn nghĩ rằng giá trị `count` sẽ tăng lên 3 sau mỗi lần click.
- **Thực tế:** Trong nhiều trường hợp, giá trị `count` có thể chỉ tăng lên 1 sau mỗi lần click, thay vì 3.
- **Giải thích:** Do tính bất đồng bộ và cơ chế gom nhóm cập nhật của React, cả ba lệnh `setCount(count + 1)` có thể được xử lý gần như đồng thời. React có thể chỉ thực hiện **một lần re-render** và sử dụng giá trị `count` **tại thời điểm trước lần cập nhật đầu tiên** cho cả ba lệnh. Điều này dẫn đến việc cập nhật không như mong đợi.
- **Vấn đề khi cập nhật State phụ thuộc vào State hiện tại:**
 - Vấn đề trên đặc biệt nghiêm trọng khi bạn muốn cập nhật state dựa trên **giá trị state hiện tại**.
 - Trong ví dụ trên, `setCount(count + 1)` đang cố gắng tăng `count` lên 1 dựa trên giá trị `count` hiện tại. Nhưng vì tính bất đồng bộ, giá trị `count` tại thời điểm cập nhật có thể không phải là giá trị mới nhất mà bạn mong đợi.

2.2 Cập nhật State với Callback Function

- **Giải pháp: Sử dụng Callback Function trong `setState` (hoặc hàm `setCount`):**
 - Để giải quyết vấn đề cập nhật state bất đồng bộ và đảm bảo cập nhật chính xác khi phụ thuộc vào state hiện tại, React cung cấp cơ chế **callback function** khi cập nhật state.
 - Thay vì truyền trực tiếp giá trị mới vào `setCount`, bạn truyền vào một **hàm**. Hàm này sẽ nhận **giá trị state trước đó** (thường được đặt tên là `prevState`, `prevCount`, `previousState`, ...) làm đối số và trả về **giá trị state mới** dựa trên `prevState`.

- **Cú pháp:**

```
setCount(prevCount => {
  // Tính toán giá trị state mới dựa trên prevCount
  return prevCount + 1;
});
```

- `setCount(prevCount => { ... })`: Thay vì `setCount(newValue)`, chúng ta truyền một hàm vào `setCount`.
- `prevCount`: Đối số đầu tiên của hàm callback, **luôn luôn** là giá trị state **mới nhất** tại thời điểm React thực hiện cập nhật. React đảm bảo giá trị `prevCount` này là chính xác, ngay cả khi có nhiều cập nhật state đang chờ xử lý.
- `return prevCount + 1;`: Hàm callback **phải trả về** giá trị state mới. Trong ví dụ này, chúng ta tính toán giá trị mới bằng cách cộng 1 vào `prevCount`.
- **Ví dụ: Sửa lại Counter với Callback Function:**

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  const handleIncrement = () => {
    setCount(prevCount => prevCount + 1); // Cập nhật lần 1 với callback
    setCount(prevCount => prevCount + 1); // Cập nhật lần 2 với callback
    setCount(prevCount => prevCount + 1); // Cập nhật lần 3 với callback
  };

  return (
    <div>
      <p>Đếm: {count}</p>
      <button onClick={handleIncrement}>Tăng 3 lần</button>
    </div>
  );
}
```



```
);  
}  
  
export default Counter;
```

- **Kết quả:** Với cách cập nhật state bằng callback function, bây giờ khi bạn click button "Tăng 3 lần", giá trị `count` sẽ **tăng lên đúng 3** sau mỗi lần click.
- **Giải thích:** Callback function đảm bảo rằng mỗi lần cập nhật state đều dựa trên **giá trị state chính xác và mới nhất** tại thời điểm đó, loại bỏ vấn đề do tính bất đồng bộ gây ra.

2.3 Khi nào nên sử dụng Callback để Update?

- **Trường hợp bắt buộc sử dụng Callback Update:**
 - **Khi giá trị state mới phụ thuộc vào giá trị state trước đó.** Đây là trường hợp chính mà callback update cần thiết. Ví dụ:
 - Tăng/giảm bộ đếm.
 - Thêm/xóa phần tử khỏi mảng state (dựa trên mảng hiện tại).
 - Cập nhật thuộc tính của một object state (dựa trên object hiện tại).
- **Trường hợp không cần (nhưng vẫn có thể) sử dụng Callback Update:**
 - **Khi giá trị state mới không phụ thuộc vào giá trị state trước đó.** Ví dụ:
 - Đặt state về một giá trị cố định, ví dụ `setCount(0)`.
 - Cập nhật state từ giá trị nhập vào input, ví dụ `setValue(event.target.value)`. Trong trường hợp này, giá trị mới (`event.target.value`) đã có sẵn và không cần dựa vào state trước đó.
- **Lưu ý:** Ngay cả khi không bắt buộc, việc sử dụng callback function khi cập nhật state **dựa trên giá trị trước đó** vẫn là một **thực hành tốt**. Nó giúp code của bạn trở nên **rõ ràng hơn, dễ hiểu hơn và an toàn hơn** trước các vấn đề tiềm ẩn liên quan đến tính bất đồng bộ.

2.4 Lợi ích của Callback Update:

- **Đảm bảo cập nhật State chính xác:** Giải quyết vấn đề cập nhật state bất đồng bộ, đặc biệt khi state mới phụ thuộc vào state cũ.
- **Tránh lỗi "race conditions":** Ngăn chặn các tình huống cập nhật state bị ghi đè hoặc mất đồng bộ khi có nhiều cập nhật xảy ra đồng thời.
- **Code dễ đọc và dễ bảo trì hơn:** Callback function thể hiện rõ ràng rằng việc cập nhật state đang dựa trên giá trị trước đó, giúp code dễ hiểu và dễ theo dõi hơn.

Render danh sách trong React

1. Mục tiêu

- Hiểu tại sao cần render danh sách động trong ứng dụng web.
- Nắm vững cách sử dụng phương thức `map()` của JavaScript để render danh sách trong React.
- Hiểu rõ tầm quan trọng của `key prop` khi render danh sách và lợi ích của nó.
- Biết cách render các loại danh sách khác nhau (mảng số, mảng chuỗi, mảng đối tượng).
- Thực hành render danh sách dữ liệu động trong component React.

2. Nội dung bài giảng

2.1 Tại sao cần Render Danh sách?

- **Dữ liệu động trong ứng dụng web:**
 - Trong hầu hết các ứng dụng web thực tế, chúng ta thường xuyên phải làm việc với dữ liệu dạng danh sách (lists). Ví dụ:
 - Danh sách sản phẩm trong trang thương mại điện tử.

- Danh sách bài viết blog.
- Danh sách người dùng trong ứng dụng quản lý.
- Danh sách công việc cần làm (todo list).
- Dữ liệu này thường **không cố định**, mà thay đổi theo thời gian (ví dụ: sản phẩm mới được thêm vào, bài viết mới được đăng, ...).
- **Render danh sách động:**
 - Để hiển thị dữ liệu danh sách động này trên giao diện người dùng, chúng ta cần có khả năng **render danh sách một cách linh hoạt và hiệu quả**.
 - Thay vì viết code HTML lặp đi lặp lại cho từng phần tử trong danh sách (cách làm thủ công và không hiệu quả), React cung cấp các cơ chế mạnh mẽ để render danh sách dữ liệu động một cách dễ dàng.
- **Ví dụ về nhu cầu render danh sách:**
 - Hãy tưởng tượng bạn muốn tạo một component hiển thị danh sách tên các loại trái cây. Khi dữ liệu trái cây có thể thay đổi, bạn không muốn phải sửa code component mỗi khi danh sách trái cây cập nhật.
 - Render danh sách động cho phép component tự động cập nhật giao diện khi dữ liệu trái cây thay đổi, giúp ứng dụng trở nên linh hoạt và dễ bảo trì hơn.

2.2 Sử dụng `map()` để Render Danh sách

- **Phương thức `map()` của JavaScript:**
 - `map()` là một phương thức rất hữu ích trong JavaScript array, được dùng để **biến đổi một mảng thành một mảng mới** bằng cách áp dụng một hàm lên từng phần tử của mảng ban đầu.
 - **Cú pháp:**

```
const newArray = originalArray.map(function(item) {  
  // Trả về giá trị đã biến đổi của 'item'  
  return /* giá trị đã biến đổi */;  
});
```

- **Kết quả:** Phương thức `map()` trả về một `newArray` chứa các giá trị đã được biến đổi.
- **`map()` trong React để Render Danh sách:**
 - Trong React, chúng ta thường sử dụng `map()` để biến đổi một mảng dữ liệu thành một mảng các phần tử JSX, sau đó hiển thị mảng JSX này trên giao diện.
 - **Ví dụ: Render danh sách trái cây sử dụng `map()`**

```
import React from 'react';  
  
function FruitList() {  
  const fruits = ['Táo', 'Chuối', 'Cam', 'Nho'];  
  
  return (  
    <ul>  
      {fruits.map(fruit => (  
        <li>{fruit}</li>  
      ))}  
    </ul>  
  );  
}  
  
export default FruitList;
```

- **Giải thích:**
 - `fruits`: Mảng dữ liệu chứa tên các loại trái cây.
 - `fruits.map(fruit => ( ... ))`: Chúng ta gọi phương thức `map()` trên mảng `fruits`.

- `fruit ⇒ ( <li>{fruit}</li> )` : Hàm callback trong `map()` nhận vào từng `fruit` (tên trái cây) và trả về một phần tử `<li>` JSX hiển thị tên trái cây đó.
- Kết quả của `fruits.map(...)` là một mảng các phần tử `<li>` JSX.
- Chúng ta đặt mảng JSX này bên trong thẻ `<ul>` để tạo thành danh sách các mục.

2.3 Tầm quan trọng của `key Prop`

- **Vấn đề khi Render Danh sách mà không có `key` :**
 - Khi bạn render một danh sách các phần tử động trong React, bạn có thể thấy cảnh báo (warning) trong console của trình duyệt, đại loại như: "Each child in a list should have a unique "key" prop." (Mỗi phần tử con trong danh sách nên có một prop "key" duy nhất).
 - Mặc dù code vẫn chạy, nhưng việc bỏ qua cảnh báo `key` có thể gây ra các vấn đề về **hiệu năng** và **hành vi không mong muốn** khi danh sách thay đổi.
- **Vai trò của `key Prop` :**
 - `key` là một **prop đặc biệt** mà bạn cần thêm vào **phần tử gốc** bên trong callback của `map()` khi render danh sách.
 - `key` giúp React **xác định duy nhất** từng phần tử trong danh sách.
 - Khi danh sách thay đổi (ví dụ: thêm, xóa, sắp xếp lại phần tử), React sử dụng `key` để **theo dõi** các phần tử:
 - React biết phần tử nào đã bị thay đổi, thêm mới hay xóa bỏ.
 - Nhờ đó, React chỉ **cập nhật (re-render) những phần tử thực sự cần thiết**, thay vì re-render lại toàn bộ danh sách.
- **Lợi ích của việc sử dụng `key Prop` :**
 - **Tối ưu hiệu năng:** Re-render hiệu quả hơn, đặc biệt với danh sách lớn hoặc khi phần tử con có component phức tạp.
 - **Cải thiện hành vi:** Đảm bảo state của các phần tử con (nếu có) được duy trì chính xác khi danh sách thay đổi. Ví dụ, nếu bạn có danh sách các ô input, `key` giúp đảm bảo input nào tương ứng với dữ liệu nào khi danh sách được sắp xếp lại.
- **Giá trị của `key Prop` :**
 - Giá trị của `key` phải là **duy nhất** giữa các phần tử **trong cùng một danh sách**.
 - Giá trị `key` nên là một **định danh ổn định** (stable identifier) cho mỗi phần tử, tức là giá trị này không thay đổi khi vị trí của phần tử trong danh sách thay đổi.
 - **Thường dùng:**
 - **ID duy nhất** từ dữ liệu (nếu có). Ví dụ: nếu bạn có danh sách sản phẩm, mỗi sản phẩm có `id` duy nhất, bạn có thể dùng `product.id` làm `key` .
 - `index` của phần tử trong mảng (chỉ nên dùng khi không có ID duy nhất và danh sách **không bao giờ bị sắp xếp lại, thêm hoặc xóa phần tử ở vị trí giữa danh sách**).
- **Ví dụ: Thêm `key Prop` vào `FruitList` :**

JavaScript

```
import React from 'react';

function FruitList() {
  const fruits = ['Táo', 'Chuối', 'Cam', 'Nho'];

  return (
    <ul>
      {fruits.map((fruit, index) => ( // Sử dụng index làm key (trong trường hợp này có thể chấp nhận)
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
}
```

```
export default FruitList;
```

- Trong ví dụ này, chúng ta sử dụng `index` làm `key` vì danh sách trái cây đơn giản và không có ID duy nhất. Tuy nhiên, **trong thực tế, bạn nên ưu tiên sử dụng ID duy nhất từ dữ liệu làm `key` nếu có thể.**

2.4 Ví dụ khác

- **Danh sách đối tượng:**

```
import React from 'react';

function ProductList() {
  const products = [
    { id: 1, name: 'Laptop', price: 1200 },
    { id: 2, name: 'Điện thoại', price: 800 },
    { id: 3, name: 'Máy tính bảng', price: 500 },
  ];

  return (
    <ul>
      {products.map(product => (
        <li key={product.id}> {/* Sử dụng product.id làm key */}
        <h3>{product.name}</h3>
        <p>Giá: ${product.price}</p>
      </li>
      )))}
    </ul>
  );
}

export default ProductList;
```

- Trong trường hợp danh sách đối tượng, chúng ta **nên sử dụng một thuộc tính duy nhất và ổn định trong đối tượng (ví dụ: `id`) làm `key`**.

Props trong React

1. Mục tiêu

- Hiểu Props là gì và vai trò của nó trong React.
- Biết cách truyền Props từ Component cha xuống Component con.
- Biết cách nhận và sử dụng Props trong Component con.
- Hiểu Prop `children` đặc biệt và cách sử dụng nó.
- Nắm vững các nguyên tắc cơ bản khi làm việc với Props (ví dụ: tính bất biến).
- Thực hành truyền và sử dụng Props trong các Component React.

2. Nội dung bài giảng

2.1 Props là gì?

- **Props (Properties) là cơ chế để truyền dữ liệu từ Component cha xuống Component con trong React** để component con có thể sử dụng và hiển thị.
- **Props là dữ liệu chỉ đọc (read-only) từ bên ngoài Component.** Điều quan trọng cần nhớ là Component con **không thể** thay đổi Props mà nó nhận được từ Component cha. Props là dữ liệu được "gửi đến" từ bên ngoài, và Component con chỉ có thể "đọc" và sử dụng chúng. Để thay đổi dữ liệu hiển thị trong Component, chúng ta sử dụng **State** (như bạn đã học ở bài trước) để quản lý dữ liệu **bên trong** Component.

- **Props giúp Component trở nên linh hoạt và tái sử dụng hơn.** Khi một Component nhận dữ liệu thông qua Props, nó có thể hiển thị nội dung khác nhau tùy thuộc vào dữ liệu Props mà nó nhận được. Điều này giúp bạn tạo ra các Component có thể tái sử dụng ở nhiều nơi khác nhau trong ứng dụng, chỉ cần truyền vào các Props khác nhau.

2.2 Tại sao cần sử dụng Props?

- **Truyền dữ liệu giữa các Component:** Như đã nói, Props là cách chính để Component cha "giao tiếp" và "truyền dữ liệu" cho Component con. Trong một ứng dụng React phức tạp, bạn sẽ thường xuyên cần truyền dữ liệu từ Component cấp cao (ví dụ: Component quản lý dữ liệu ứng dụng) xuống các Component cấp thấp hơn (ví dụ: Component hiển thị giao diện người dùng).
- **Tạo Component động và tái sử dụng:** Props cho phép bạn tạo ra các Component có thể hiển thị nội dung khác nhau dựa trên dữ liệu được truyền vào. Ví dụ, bạn có thể tạo một Component `Card` chung để hiển thị thông tin sản phẩm, và sau đó tái sử dụng `Card` này để hiển thị thông tin của nhiều sản phẩm khác nhau, chỉ cần truyền vào Props khác nhau cho mỗi lần sử dụng.
- **Xây dựng giao diện người dùng phức tạp từ các Component nhỏ:** React khuyến khích việc chia nhỏ giao diện người dùng thành các Component nhỏ, độc lập và tái sử dụng. Props là cầu nối để kết nối các Component nhỏ này lại với nhau, tạo thành một giao diện người dùng phức tạp và có cấu trúc.

2.3 Cách truyền Props từ Component cha xuống Component con

Để truyền Props từ Component cha xuống Component con, bạn sử dụng cú pháp tương tự như khi bạn truyền thuộc tính (attributes) cho các thẻ HTML trong JSX.

- **Ví dụ: Truyền Props cho Component `Counter`**

Hãy tạo một Component cha tên là `App` và truyền Props cho Component `Counter` mà chúng ta đã tạo ở bài trước.

```
// src/App.jsx
import './App.css';
import Counter from './Counter';

function App() {
  const initialCount = 10; // Dữ liệu Props mà chúng ta muốn truyền

  return (
    <div className="App">
      <h1>Ứng dụng Counter</h1>
      <Counter startCount={initialCount} /> {/* Truyền Prop 'startCount' */}
    </div>
  );
}

export default App;
```

2.4 Cách nhận và sử dụng Props trong Component con

Trong Component con, Props được truyền vào dưới dạng một **đối số (argument) cho Function Component**. Đối số này thường được đặt tên là `props` (quy ước), nhưng bạn có thể đặt tên khác nếu muốn.

- **Ví dụ: Nhận và sử dụng Prop `startCount` trong Component `Counter`**

Hãy sửa đổi Component `Counter.jsx` để nhận và sử dụng Prop `startCount` mà chúng ta đã truyền từ Component `App`.

JavaScript

```
// src/Counter.jsx
import React, { useState } from 'react';
import styles from './Counter.css';

// Component Counter nhận Props dưới dạng đối số 'props'
function Counter(props) {
```

```
// Sử dụng Prop 'startCount' từ props để khởi tạo state 'count'
const [count, setCount] = useState(props.startCount);

const handleIncrement = () => {
  setCount(prevCount => prevCount + 1);
};

return (
  <div className={styles.counter}>
    <p className={styles.count}>Đếm: {count}</p>
    <button className={styles.button} onClick={handleIncrement}>Tăng</button>
  </div>
);
}

export default Counter;
```

Giải thích:

- `function Counter(props) { ... }` : Chúng ta khai báo Component `Counter` là một Function Component và nó nhận một đối số là `props` . Đối số `props` là một **object JavaScript**, chứa tất cả các Props được truyền xuống từ Component cha.
- `const [count, setCount] = useState(props.startCount);` : Trong Component `Counter` , chúng ta truy cập vào Prop `startCount` thông qua `props.startCount` . Chúng ta sử dụng giá trị `props.startCount` để khởi tạo giá trị ban đầu cho state `count` bằng Hook `useState` .

Bây giờ, khi bạn chạy ứng dụng, bạn sẽ thấy Component `Counter` hiển thị "Đếm: 10" (giá trị của `initialCount` mà chúng ta truyền từ Component `App`) thay vì "Đếm: 1" như trước. Giá trị ban đầu của bộ đếm đã được "động hóa" thông qua Props.

• Truy cập Props bằng destructuring (ES6 Destructuring)

Trong ví dụ trên, chúng ta truy cập Prop `startCount` bằng cách sử dụng `props.startCount` . Tuy nhiên, nếu Component của bạn nhận nhiều Props, việc truy cập `props.prop1` , `props.prop2` , `props.prop3` , ... có thể trở nên dài dòng và kém rõ ràng.

Để code trở nên gọn gàng và dễ đọc hơn, bạn có thể sử dụng **ES6 Destructuring** để "giải nén" các Props trực tiếp từ đối số `props` .

Ví dụ: Sử dụng destructuring để nhận Prop `startCount`

JavaScript

```
// src/Counter.jsx
import React, { useState } from 'react';
import styles from './Counter.css';

// Component Counter nhận Prop 'startCount' bằng destructuring
function Counter({ startCount }) {
  // Sử dụng trực tiếp biến 'startCount' để khởi tạo state 'count'
  const [count, setCount] = useState(startCount);

  const handleIncrement = () => {
    setCount(prevCount => prevCount + 1);
  };

  return (
    <div className={styles.counter}>
      <p className={styles.count}>Đếm: {count}</p>
      <button className={styles.button} onClick={handleIncrement}>Tăng</button>
    </div>
  );
}
```

```
export default Counter;
```

Giải thích:

- `function Counter({ startCount }) { ... }` : Thay vì `function Counter(props) { ... }` , chúng ta sử dụng cú pháp destructuring trực tiếp trong định nghĩa hàm: `{ { startCount } }` . Điều này có nghĩa là chúng ta đang "giải nén" Prop `startCount` từ object `props` và tạo ra một biến cục bộ có tên là `startCount` trong phạm vi hàm `Counter` .
- `const [count, setCount] = useState(startCount);` : Bây giờ, chúng ta có thể sử dụng trực tiếp biến `startCount` (thay vì `props.startCount`) để khởi tạo state `count` . Code trở nên ngắn gọn và dễ đọc hơn.

Destructuring là một kỹ thuật rất hữu ích và thường được sử dụng khi làm việc với Props trong React.

2.5 Truyền nhiều Props và các loại dữ liệu Props khác nhau

Bạn có thể truyền nhiều Props cho một Component, và Props có thể chứa nhiều loại dữ liệu khác nhau, không chỉ là số như trong ví dụ trên.

• **Ví dụ: Truyền nhiều Props với các loại dữ liệu khác nhau**

Hãy sửa đổi Component `App` để truyền thêm Props cho Component `Counter` , bao gồm:

- `incrementBy` : Số lượng tăng mỗi khi button "Tăng" được click (kiểu số).
- `label` : Nhãn cho button "Tăng" (kiểu chuỗi).
- `isHighlight` : Xác định xem có highlight Component `Counter` hay không (kiểu boolean).
- `fruits` : Danh sách trái cây yêu thích (kiểu mảng).

JavaScript

```
// src/App.jsx
import './App.css';
import Counter from './Counter';

function App() {
  const initialCount = 5;
  const incrementValue = 2;
  const buttonLabel = "Tăng gấp đôi";
  const highlight = true;
  const favoriteFruits = ["Táo", "Chuối", "Cam"];

  return (
    <div className="App">
      <h1>Ứng dụng Counter</h1>
      <Counter
        startCount={initialCount}
        incrementBy={incrementValue}
        label={buttonLabel}
        isHighlight={highlight}
        fruits={favoriteFruits}
      />
    </div>
  );
}

export default App;
```

Và sửa đổi Component `Counter` để nhận và sử dụng các Props mới này:

```
// src/Counter.jsx
import React, { useState } from 'react';
import styles from './Counter.css';
```



```
function Counter({ startCount, incrementBy, label, isHighlight, fruits }) {
  const [count, setCount] = useState(startCount);

  const handleIncrement = () => {
    // Tăng theo giá trị 'incrementBy' từ Props
    setCount(prevCount => prevCount + incrementBy);
  };

  return (
    <div className={` ${styles.counter} ${isHighlight ? styles.highlight : ''} `> {/* Sử dụng Prop 'isHighlight' đ
      <p className={styles.count}>Đếm: {count}</p>
      <button className={styles.button} onClick={handleIncrement}>
        {label} {/* Sử dụng Prop 'label' cho nhãn button */}
      </button>
      <p>Trái cây yêu thích:</p>
      <ul>
        {fruits.map((fruit, index) => (
          <li key={index}>{fruit}</li> {/* Render danh sách 'fruits' từ Props */}
        ))}
      </ul>
    </div>
  );
}

export default Counter;
```

Giải thích:

- **Component App** : Chúng ta truyền nhiều Props cho Component `Counter` , với các loại dữ liệu khác nhau: số (`startCount` , `incrementBy`), chuỗi (`label`), boolean (`isHighlight`), và mảng (`fruits`).
- **Component Counter** :
 - `function Counter({ startCount, incrementBy, label, isHighlight, fruits }) { ... }` : Chúng ta sử dụng destructuring để nhận tất cả các Props được truyền vào.
 - `setCount(prevCount => prevCount + incrementBy);` : Hàm `handleIncrement` bây giờ sử dụng Prop `incrementBy` để tăng giá trị đếm.
 - `<button className={styles.button} onClick={handleIncrement}>{label}</button>` : Nhãn của button "Tăng" được lấy từ Prop `label` .
 - `<div className={ ` ${styles.counter} ${isHighlight ? styles.highlight : ''} `}>` : Chúng ta sử dụng conditional styling để thêm class `styles.highlight` vào `<div>` cha của `Counter` nếu Prop `isHighlight` là `true` . (Bạn cần định nghĩa class `.highlight` trong `Counter.module.css` nếu muốn thấy hiệu ứng highlight).
 - Render danh sách trái cây yêu thích `fruits` từ Props bằng phương thức `map()` .

Ví dụ này minh họa cách Props có thể được sử dụng để truyền dữ liệu đa dạng và tùy chỉnh giao diện và hành vi của Component con dựa trên dữ liệu được truyền từ Component cha.

2.6 Prop `children` đặc biệt

Trong React, có một Prop đặc biệt có tên là `children` . Prop `children` cho phép bạn truyền **JSX (bao gồm cả các Component khác)** làm **nội dung con** của một Component.

- **Ví dụ: Sử dụng Prop `children` để tạo Component `Card` đa năng**

Hãy tạo một Component `Card` đơn giản để bao bọc nội dung khác nhau và có thể tái sử dụng.

JavaScript

```
// src/Card.jsx
import React from 'react';
import styles from './Card.module.css';

function Card(props) {
```



```
return (  
  <div className={styles.card}>  
    {props.children} {/* Hiển thị nội dung con được truyền qua Prop 'children' */}  
  </div>  
);  
}  
  
export default Card;
```

Giải thích:

- `function Card(props) { ... }` : Component `Card` nhận Props (thường đặt tên là `props`).
- `{props.children}` : Chúng ta hiển thị `props.children` bên trong thẻ `<div>` của `Card` . `props.children` sẽ chứa bất kỳ nội dung JSX nào được đặt giữa thẻ mở và thẻ đóng của Component `Card` khi sử dụng nó.

Bây giờ, hãy sử dụng Component `Card` trong `App.jsx` và truyền nội dung con khác nhau cho nó:

```
// src/App.jsx  
import './App.css';  
import Counter from './Counter';  
import Card from './Card'; // Import Component Card  
  
function App() {  
  const initialCount = 5;  
  const incrementValue = 2;  
  const buttonLabel = "Tăng gấp đôi";  
  const highlight = true;  
  const favoriteFruits = ["Táo", "Chuối", "Cam"];  
  
  return (  
    <div className="App">  
      <h1>Ứng dụng Counter</h1>  
  
      <Card> {/* Sử dụng Component Card và truyền nội dung con */}  
        <h2>Counter 1</h2>  
        <Counter  
          startCount={initialCount}  
          incrementBy={incrementValue}  
          label={buttonLabel}  
          isHighlight={highlight}  
          fruits={favoriteFruits}  
        />  
      </Card>  
  
      <Card> {/* Sử dụng Component Card lần nữa với nội dung con khác */}  
        <h2>Counter 2</h2>  
        <p>Đây là một Counter khác bên trong Card.</p>  
        <Counter startCount={0} incrementBy={1} label="Tăng" isHighlight={false} fruits=[] />  
      </Card>  
    </div>  
  );  
}  
  
export default App;
```

Giải thích:

- `<Card> ... </Card>` : Khi chúng ta sử dụng Component `Card` như thế này, mọi thứ được viết **giữa thẻ mở `<Card>` và thẻ đóng `</Card>`** sẽ được truyền vào Component `Card` thông qua Prop `children` .
- Trong Component `Card` , `{props.children}` sẽ hiển thị toàn bộ nội dung con mà chúng ta đã truyền vào.

Với Prop `children` , Component `Card` trở thành một Component **đa năng và tái sử dụng**. Bạn có thể sử dụng `Card` để bao bọc bất kỳ nội dung nào bạn muốn (văn bản, hình ảnh, Component khác, v.v.) và `Card` sẽ lo việc hiển thị khung bao bọc chung (ví dụ: style, layout).

2.7 Nguyên tắc quan trọng khi làm việc với Props: Props là bất biến (Immutable)

- **Props là dữ liệu chỉ đọc (read-only) từ bên ngoài Component.** Như đã nhấn mạnh ở trên, Component con **không được phép** thay đổi Props mà nó nhận được từ Component cha.
- **Tính bất biến (immutability) của Props là một nguyên tắc quan trọng trong React.** Điều này có nghĩa là khi bạn muốn thay đổi dữ liệu dựa trên Props, bạn không được sửa đổi trực tiếp object Props. Thay vào đó, bạn cần tạo ra một bản sao mới của dữ liệu và thực hiện thay đổi trên bản sao đó, hoặc tốt nhất là yêu cầu Component cha thay đổi Props (nếu cần thiết).
- **Tại sao Props phải bất biến?**
 - **Đảm bảo tính nhất quán và dễ dự đoán:** Việc Props là bất biến giúp đảm bảo rằng dữ liệu của Component luôn nhất quán và dễ dự đoán. Bạn có thể tin tưởng rằng Props mà Component nhận được sẽ không bị thay đổi bất ngờ từ bên trong Component đó.
 - **Tối ưu hiệu năng:** React có thể tối ưu hiệu năng render dựa trên việc so sánh Props và State. Khi Props là bất biến, React có thể dễ dàng xác định khi nào Component cần re-render dựa trên việc so sánh tham chiếu của Props.
 - **Tuân thủ luồng dữ liệu một chiều (one-way data flow) của React:** React đi theo luồng dữ liệu một chiều, dữ liệu luôn đi từ Component cha xuống Component con thông qua Props. Việc Props là bất biến giúp duy trì luồng dữ liệu này và làm cho ứng dụng dễ quản lý và debug hơn.
- **Ví dụ minh họa việc không được sửa đổi Props:**

```
// Ví dụ KHÔNG ĐÚNG - KHÔNG SỬA ĐỔI PROPS TRỰC TIẾP
function WrongCounter(props) {
  // KHÔNG NÊN LÀM NHƯ VẬY - SỬA ĐỔI PROPS TRỰC TIẾP LÀ SAI NGUYÊN TẮC
  props.startCount = props.startCount + 1; // CỐ GẮNG SỬA ĐỔI PROP 'startCount'

  const [count, setCount] = useState(props.startCount); // Sử dụng (giá trị đã bị sửa đổi) của prop

  const handleIncrement = () => {
    setCount(prevCount => prevCount + 1);
  };

  return (
    <div>
      <p>Đếm: {count}</p>
      <button onClick={handleIncrement}>Tăng</button>
    </div>
  );
}
```

Trong ví dụ trên, chúng ta **cố gắng sửa đổi trực tiếp Prop** `props.startCount` . Đây là một hành động **sai nguyên tắc** trong React và có thể dẫn đến các hành vi không mong muốn và khó debug. Bạn **không bao giờ** được sửa đổi trực tiếp Props.

- **Cách làm đúng nếu muốn thay đổi dữ liệu dựa trên Props:**

Nếu bạn muốn thay đổi dữ liệu dựa trên Props, bạn nên sử dụng **State** để quản lý dữ liệu bên trong Component. Như ví dụ Component `Counter` ban đầu, chúng ta đã sử dụng Prop `startCount` để **khởi tạo giá trị ban đầu cho State** `count` . Sau đó, mọi thay đổi về dữ liệu (ví dụ: khi click button "Tăng") sẽ được thực hiện thông qua việc cập nhật **State** `count` , chứ không phải sửa đổi trực tiếp Prop `startCount` .

3. Tổng kết về Props:

- Props là cơ chế truyền dữ liệu **một chiều từ Component cha xuống Component con**.
- Props giúp Component trở nên **linh hoạt, tái sử dụng và dễ cấu thành**.

- Props là **dữ liệu chỉ đọc (read-only)** từ bên ngoài Component, Component con không được phép sửa đổi Props.
 - Sử dụng **destructuring** để truy cập Props một cách gọn gàng.
 - Prop `children` cho phép truyền **JSX làm nội dung con** của Component.
 - Luôn tuân thủ nguyên tắc **Props là bất biến (immutable)**.
-

Giảng viên đưa file **Tổng Hợp Kiến Thức** cho học viên, và hướng dẫn làm **Bài Tập Cuối Chương**.